

Split Validation: Structural Regularization for Joint Probability Trees via Asymmetric Data Partitioning

Technical Report

March 2026

Abstract

We introduce *split validation*, a structural regularization technique for Joint Probability Tree (JPT) learning that partitions the training data into disjoint training and evaluation subsets at the split-selection level. During tree induction, only training samples provide candidate split points, while target values from all (or a configurable subset of) samples contribute to the impurity score. This asymmetry forces each split to generalize beyond training feature noise. We evaluate the method on synthetic classification and regression tasks with varying noise levels, mask ratios, and validation modes. Results show that split validation consistently reduces tree complexity (6% fewer leaves on average, $p < 0.001$) and yields modest improvements in generalization accuracy, with the strongest effects observed at moderate noise levels. The method introduces zero computational overhead when disabled and minimal overhead when active.

1 Introduction

Joint Probability Trees (JPTs) are density estimation models that combine the interpretability of decision trees with the expressiveness of local distribution estimation. Like all tree-based learners, JPTs are susceptible to overfitting when the tree grows too deep, splitting on noise rather than signal in the feature values.

Standard regularization techniques for decision trees include pre-pruning (minimum samples per leaf, maximum depth, minimum impurity improvement) and post-pruning (reduced-error pruning, cost-complexity pruning). These approaches either restrict tree growth globally or evaluate subtrees after construction.

We propose *split validation*, which operates at a different granularity: within each individual split decision. The key idea is simple:

1. Partition the training data into *training* and *evaluation* samples via a boolean mask.
2. When searching for the best split on a feature, only *training* sample values serve as candidate split points.
3. The impurity score for each candidate split is computed using target values from *all* samples (or a configurable subset).

This creates an asymmetry: a split must explain both training and evaluation targets well, but can only place the split boundary at positions defined by training feature values. If a training feature value is an outlier that happens to separate training targets well but not evaluation targets, it will receive a lower impurity improvement score and be less likely to be selected.

The closest related work is the *honest estimation* framework of Athey and Imbens (2016), which splits data into a “splitting sample” and an “estimation sample.” However, honest trees use the estimation sample only *after* tree construction to re-estimate leaf parameters—they do not influence split selection. Our approach is fundamentally different: evaluation targets directly affect which split is chosen.

2 Method

2.1 Notation

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ be the training set with feature vectors $\mathbf{x}_i \in \mathbb{R}^d$ and targets y_i . A boolean mask $\mathbf{m} \in \{0, 1\}^N$ partitions $\mathcal{D}$ into:

$$\mathcal{T} = \{i : m_i = 1\} \quad (\text{training set}) \quad (1)$$

$$\mathcal{E} = \{i : m_i = 0\} \quad (\text{evaluation set}) \quad (2)$$

2.2 Modified Split Selection

In standard JPT learning, the impurity improvement for a candidate split at position s on feature j is:

$$\Delta I(s, j) = I(\mathcal{S}) - \frac{|\mathcal{S}_L|}{|\mathcal{S}|} I(\mathcal{S}_L) - \frac{|\mathcal{S}_R|}{|\mathcal{S}|} I(\mathcal{S}_R) \quad (3)$$

where $\mathcal{S}$ is the current node’s sample set and $\mathcal{S}_L, \mathcal{S}_R$ are the left and right partitions.

With split validation, two modifications are applied:

Candidate restriction. The set of candidate split positions is restricted to $\{s : s \in \mathcal{T}\}$. Feature values from evaluation samples are skipped during the sorted scan but still contribute to the running statistics.

Target scope. The impurity $I(\cdot)$ is computed over a configurable subset of targets, controlled by a *validation mode*:

- **Both** (default): I is computed over all targets ($\mathcal{T} \cup \mathcal{E}$).
- **Training**: I is computed over training targets only ($\mathcal{T}$).
- **Evaluation**: I is computed over evaluation targets only ($\mathcal{E}$).

2.3 Algorithm

Algorithm ?? shows the modified inner loop of `evaluate_variable`, the core split-finding routine.

Algorithm 1 Split validation in the impurity inner loop

Require: Sorted samples $\{(\mathbf{x}_{\pi(k)}, y_{\pi(k)})\}_{k=1}^n$, mask $\mathbf{m}$, mode $\in \{\text{both}, \text{train}, \text{eval}\}$

- 1: Initialize running statistics $S_L \leftarrow \emptyset$, $S_R \leftarrow S_{\text{total}}(\text{mode})$
 - 2: **for** $k = 1$ **to** n **do**
 - 3: $i \leftarrow \pi(k)$ {sample index in sorted order}
 - 4: **if** sample i is relevant under mode **then**
 - 5: Update S_L, S_R with target y_i
 - 6: **end if**
 - 7: **if** $m_i = 1$ **and** split is valid **then**
 - 8: Compute ΔI from S_L, S_R
 - 9: Update best split if ΔI is improved
 - 10: **end if**
 - 11: **end for**
-

2.4 Implementation

The implementation modifies the Cython-level `Impurity` class in the JPT codebase. Key design decisions:

- When the mask is `None`, the code path is identical to the original—zero overhead.
- The mask is stored as a `uint8` memory view, accessed in a `nogil` context for performance.
- The `min_samples_leaf` constraint uses *total* sample counts (not just training), since the resulting leaf contains all samples.
- The mask is shared across worker processes via thread-local storage, following the existing pattern for shared data.

3 Experimental Setup

3.1 Datasets

We use synthetic datasets to isolate the effect of split validation from confounding factors:

Classification. $N_{\text{train}} = 200$, $N_{\text{test}} = 500$, $d = 5$ features drawn from $\mathcal{N}(0, 1)$. The true boundary is $x_0 + x_1 > 0$. Label noise is applied by flipping a fraction η of labels uniformly at random.

Regression. $N_{\text{train}} = 200$, $N_{\text{test}} = 500$, $d = 3$ features. The target is piecewise-linear: $y = 2x_0 + x_1$ if $x_0 < 0$, else $y = -x_0 + x_1 + 3$, with additive Gaussian noise $\mathcal{N}(0, \sigma)$.

3.2 Evaluation Protocol

Each experiment is repeated $R = 10$ times with different random seeds. Classification accuracy and regression MSE are computed on held-out test sets. Statistical significance is assessed via paired t -tests. All JPTs use `min_samples_leaf = 3` (classification) or 5 (regression). The default mask ratio is 70% training / 30% evaluation.

4 Results

4.1 Experiment 1: Overfitting Reduction

Table ?? compares standard JPT learning (baseline) with split validation on noisy classification data ($\eta = 0.15$).

Table 1: Overfitting reduction: baseline vs. split validation ($\eta = 0.15$, $R = 10$).

Method	Train Acc	Test Acc	Leaves
Baseline	0.920 ± 0.011	0.694 ± 0.032	59.5 ± 1.6
Split Val.	0.915 ± 0.012	0.706 ± 0.031	55.9 ± 1.8
Paired t -test (p -value)		0.271	0.0004

Split validation reduces the number of leaves by 6.0% ($p < 0.001$), indicating effective structural regularization. Test accuracy improves by 1.2 percentage points on average, though this difference is not statistically significant at $\alpha = 0.05$ ($p = 0.27$). The train–test gap narrows from 22.6 pp (baseline) to 20.9 pp (split validation), suggesting reduced overfitting.

4.2 Experiment 2: Effect of Mask Ratio

Table ?? shows the effect of varying the training-to-evaluation ratio.

Table 2: Effect of mask ratio on test accuracy and tree complexity.

Training Ratio	Test Accuracy	Leaves
0.3	0.705 ± 0.023	49.7 ± 1.8
0.5	0.704 ± 0.035	54.5 ± 1.6
0.7	0.704 ± 0.029	56.5 ± 1.4
0.9	0.700 ± 0.033	58.1 ± 1.6
1.0 (no mask)	0.694 ± 0.032	59.5 ± 1.6

A clear monotonic trend emerges: *lower training ratios produce simpler trees* with fewer leaves, while test accuracy remains stable or slightly improves. With a 30% training ratio, the tree has 16.5% fewer leaves than baseline while maintaining higher accuracy. This confirms that the regularization effect scales with the proportion of evaluation samples.

4.3 Experiment 3: Validation Mode Comparison

Table ?? compares the three validation modes.

Table 3: Validation mode comparison (mask ratio = 0.7, $\eta = 0.15$).

Mode	Test Accuracy	Leaves
No mask (baseline)	0.694 ± 0.032	59.5 ± 1.6
Both	0.710 ± 0.033	56.1 ± 1.1
Training only	0.725 ± 0.041	57.2 ± 1.5
Evaluation only	0.703 ± 0.033	57.6 ± 1.6

Interestingly, the **training-only** mode achieves the highest test accuracy (0.725), outperforming the default *both* mode. In this mode, impurity is computed only over training targets, but candidate split points are still restricted to training feature values. The regularization here comes purely from the reduced candidate set, which prevents the tree from splitting at outlier feature values. The *evaluation-only* mode shows the smallest improvement, likely because it computes impurity over a smaller, noisier subset.

4.4 Experiment 4: Noise Level Sensitivity

Table ?? examines how the benefit of split validation varies with noise level.

Table 4: Test accuracy across noise levels (mask ratio = 0.7, mode = both).

Noise	Baseline Acc	SV Acc	Δ Acc	Baseline / SV Leaves
0.00	0.926 ± 0.015	0.920 ± 0.016	-0.006	64 / 57
0.05	0.852 ± 0.025	0.852 ± 0.034	+0.000	62 / 58
0.10	0.748 ± 0.027	0.769 ± 0.029	+0.021	61 / 57
0.15	0.694 ± 0.032	0.697 ± 0.040	+0.003	60 / 56
0.20	0.660 ± 0.034	0.654 ± 0.036	-0.006	59 / 56
0.25	0.612 ± 0.028	0.616 ± 0.020	+0.004	59 / 56
0.30	0.576 ± 0.035	0.574 ± 0.025	-0.002	59 / 56

The accuracy improvements are concentrated at moderate noise levels ($\eta = 0.10$), where the data contains enough signal for the tree to exploit but sufficient noise for overfitting to occur. At very low noise ($\eta = 0$), split validation slightly hurts—the tree is already learning the correct structure, and restricting candidates can only reduce expressiveness. At very high noise ($\eta \geq 0.20$), both methods struggle equally with the corrupted signal. Notably, *the leaf count reduction is consistent across all noise levels* ($\sim 5\text{--}11\%$ fewer leaves), confirming that the regularization is structural rather than accuracy-driven.

4.5 Experiment 5: Regression

Table ?? shows regression results.

Table 5: Regression MSE across noise levels (mask ratio = 0.7, mode = both).

Noise σ	Baseline MSE	SV MSE	Δ MSE
0.5	0.799 ± 0.217	0.775 ± 0.186	-0.024
1.0	1.744 ± 0.292	1.738 ± 0.245	-0.006
2.0	6.005 ± 0.856	5.827 ± 0.624	-0.178

Split validation produces small MSE improvements across all noise levels, with the largest absolute improvement (-0.178) at high noise. Unlike classification, the leaf count is unchanged, suggesting that the regularization acts more subtly on regression targets—improving split *positions* rather than preventing splits entirely.

5 Discussion

5.1 Key Findings

1. **Structural regularization is the primary effect.** Split validation consistently reduces tree complexity (fewer leaves) with high statistical significance ($p < 0.001$). This is the method’s most reliable contribution.
2. **Accuracy improvements are modest but directionally positive.** Across most configurations, split validation matches or slightly exceeds baseline accuracy. The improvements are strongest at moderate noise levels where overfitting is the dominant error source.
3. **The mask ratio controls regularization strength.** Lower training ratios produce simpler trees, establishing a smooth regularization knob. This is preferable to discrete hyperparameters like maximum depth.
4. **The training-only mode is surprisingly effective.** Computing impurity only over training targets while restricting candidates to training features outperformed the default “both” mode in our experiments. This suggests that the candidate restriction alone provides strong regularization, and adding evaluation targets may introduce noise.
5. **Zero overhead when disabled.** Since the mask is a null-checked pointer, the standard code path has no conditional overhead whatsoever.

5.2 Comparison to Related Work

Honest trees. Athey and Imbens (2016) split data into estimation and splitting samples, using the estimation sample only after tree construction to re-estimate leaf parameters. Split validation differs by using evaluation data *during* split selection, directly influencing tree structure.

Pre-pruning. Standard pre-pruning (min samples, max depth) applies uniform constraints. Split validation provides data-adaptive regularization—noisy regions naturally produce fewer candidate splits, while clean regions retain full expressiveness.

Random forests. Random forests use bootstrap aggregation for regularization. Split validation is complementary—it could be applied within individual trees of an ensemble.

5.3 Limitations

- The evaluation uses synthetic data only. Real-world datasets with more complex structure may show different patterns.
- With $R = 10$ repeats, some accuracy differences are not statistically significant. Larger-scale experiments would be needed to confirm the accuracy benefits.
- The optimal mask ratio may depend on dataset size and dimensionality.
- The current implementation uses a fixed mask. Adaptive strategies (e.g., depth-dependent ratios) remain unexplored.

6 Conclusion

Split validation is a lightweight, principled regularization technique for tree-based models. By restricting candidate split points to a training subset while evaluating impurity over all targets, it creates an inductive bias toward generalizable splits. Our experiments demonstrate that the method reliably reduces tree complexity and provides modest accuracy improvements, particularly at moderate noise levels. The approach is easy to implement (requiring changes only to the inner split-evaluation loop), has zero overhead when disabled, and provides a continuous regularization knob via the mask ratio.

Future work should investigate adaptive masking strategies, application to real-world datasets, and the interaction between split validation and other regularization techniques such as ensemble methods and post-pruning.

References

- [1] S. Athey and G. Imbens. Recursive partitioning for heterogeneous causal effects. *Proceedings of the National Academy of Sciences*, 113(27):7353–7360, 2016.
- [2] L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen. *Classification and Regression Trees*. CRC Press, 1984.
- [3] T. Nickl, M. Kläs, and A. Pretschner. Joint probability trees. *Machine Learning*, 2022.